INFORME TÉCNICO PATRONES ESTRUCTURALES PROXY Y FACADE EN AGROLAB SINÚ

Integrantes:

Efrén Polo
Fabián Salgado
Juan Villalobos

Docente:

Alberto Paternina León

Facultad de Ingeniería
PROGRAMA INGENIERÍA DE SISTEMAS
Departamento de ingenierías de sistemas y telecomunicaciones.
Universidad de Córdoba, Montería.

Programación III - Grupo 4
Patrones Asignados: Proxy + Facade
Repositorio GitHub: https://github.com/TheGhartty/Taller-Programacion

1. Introducción

El presente informe técnico expone la evaluación, diagnóstico y refactorización del sistema de monitoreo agrológico AgroLab Sinú mediante la aplicación de los patrones de diseño estructurales Proxy y Facade. Inicialmente, la plataforma presentaba fallos severos de seguridad, redundancia de consultas costosas a bases de datos distribuidas y omisión de reglas de negocio críticas por alto acoplamiento en clientes móviles y web. La refactorización mediante un proxy de protección/caché y una fachada centralizada permitió estabilizar el sistema, garantizar la facturación obligatoria de servicios y optimizar el rendimiento global.

2. Dominio AgroLab Sinú y Necesidades de Arquitectura

AgroLab Sinú procesa muestras edafológicas y de laboratorio en el valle del río Sinú. Para garantizar un funcionamiento seguro y rentable, el dominio exige un control estricto de acceso por roles y la orquestación confiable de sus subsistemas internos.

Necesidad del Negocio	Problema Detectado	Patrón Solución
Seguridad y Caché de Muestras	Acceso no autorizado a muestras privadas y consultas redundantes a bases de datos en la nube.	Patrón Proxy (Protección + Caché + Lazy Init)
Orquestación de Procesos de Laboratorio	Omisión accidental de facturación y complejidad en clientes al coordinar 5 subsistemas.	Patrón Facade (Fachada Unificada)

3. Patrón Proxy

3.1 Versión Inicial y Defectos

En la versión inicial (paquete `proxy.inicial`), la clase `ServicioResultados` ejecutaba directa y costosamente las consultas. Se identificaron dos defectos críticos: el **DEFECTO 1** permitió que un usuario 'Anonimo' consultara la muestra privada 'M-002' sin restricción alguna; el **DEFECTO 2** ocurrió cuando el usuario 'Efren' consultó dos veces seguidas la muestra 'M-001', forzando ejecuciones reiteradas de `consultarResultado()` contra la base de datos por ausencia de almacenamiento en caché.

3.2 Versión Refactorizada

En el paquete `proxy.refactorizado`, se implementó la clase `ServicioResultadosProxy` respaldada por roles de usuario (ANALISTA, AGRICULTOR, VISITANTE). Cada petición invoca en primer lugar el método privado `tieneAcceso()`, generando auditoría y lanzando `SecurityException` si no es autorizado. Si la verificación es exitosa, valida la caché local antes de inicializar perezosamente (lazy initialization) el sujeto real `ServicioResultadosReal`.

3.3 Beneficios y Limitaciones del Proxy

Beneficios	Limitaciones
Protección transparente y control de acceso robusto por roles sin alterar la interfaz original.	Incrementa la latencia en la primera llamada debido al procesamiento previo de seguridad.

Beneficios	Limitaciones
Reducción drástica del tráfico de red y costo computacional mediante caché local.	Riesgo de entregar datos desactualizados si no se gestiona una política de invalidación de caché.

4. Patrón Facade

4.1 Versión Inicial y Defectos

En la solución inicial (paquete `facade.inicial`), los clientes coordinaban manualmente 5 subsistemas: `RecepcionMuestras`, `LaboratorioQuimico`, `GeneradorInforme`, `Facturacion` y `NotificadorSMS`. Esto provocó que en el método `flujoAppMovil()` el desarrollador omitiera invocar la clase `Facturacion`, entregando el informe al usuario sin generar el cobro del servicio.

4.2 Versión Refactorizada

Se introdujo la clase `AgroLabFacade` en el paquete `facade.refactorizado`, estableciendo la constante `TARIFA_ANALISIS_COMPLETO = 85000.0`. El método unificado `solicitarAnalisisCompleto()` orquesta el registro, mediciones, informe, facturación obligatoria e inclusión de SMS. Adicionalmente, se dispuso de `solicitarMedicionRapidaPH()` para flujos simplificados.

4.3 Beneficios y Limitaciones de Facade

Beneficios	Limitaciones
Desacopla a los clientes de la complejidad interna e inmuniza la ejecución de reglas de negocio.	Puede convertirse en un "Objeto Dios" si acumula responsabilidades ajenas a la orquestación.
Garantiza la facturación y notificación estandarizada en todos los puntos de entrada del sistema.	No impide que un cliente mal intencionado salte la fachada si los subsistemas permanecen públicos.

Aunque Proxy y Facade actúan como intermediarios entre el cliente y los objetos finales, sus intenciones arquitecturales difieren radicalmente. Proxy se enfoca en controlar el acceso a un único objeto compartiendo exactamente su misma interfaz, mientras que Facade define una interfaz completamente nueva para simplificar la interacción con múltiples subsistemas.

5. Diagramas UML Explicados

Criterio	Patrón Proxy	Patrón Facade
Propósito Principal	Controlar acceso, lazy loading y almacenamiento en caché.	Simplificar y orquestar una interfaz compleja de varios subsistemas.
Relación / Cardinalidad	Relación 1 a 1 (Proxy -> Sujeto Real).	Relación 1 a N (Facade -> Subsistemas N).
Interfaz Presentada	Idéntica a la del sujeto real (ServicioResultados).	Nueva interfaz simplificada (AgroLabFacade).
Caso en AgroLab	ServicioResultadosProxy (Seguridad por ROL + Caché).	AgroLabFacade (Orquestación del flujo completo de laboratorio).

UML Proxy: Muestra que `ServicioResultadosProxy` y `ServicioResultadosReal` implementan la interfaz `ServicioResultados`. El Proxy agrega una asociación hacia `Usuario` para validar roles y almacena en caché los datos antes de derivar la llamada al sujeto real.

UML Facade: Ilustra a `AgroLabFacade` como único punto de entrada para los clientes web/móvil, manteniendo asociaciones privadas hacia los 5 subsistemas e impidiendo omisiones en el flujo operativo.

6. Punto Adicional JDK / Spring Framework

En la API estándar de Java, el patrón Proxy se manifiesta en `java.lang.reflect.Proxy`, permitiendo la creación de proxies dinámicos en tiempo de ejecución para interceptar métodos e inyectar aspectos de seguridad o transacciones (base de Spring AOP). Por su parte, el patrón Facade se observa en la clase `java.nio.file.Files`, la cual ofrece una interfaz simplificada para operaciones complejas sobre el sistema de archivos (lectura, escritura, atributos), ocultando la complejidad de los proveedores de FileSystem subyacentes.

7. Conclusiones y Referencias

Conclusiones:

1. La refactorización mediante el patrón Proxy garantizó la seguridad en AgroLab Sinú al anteponer la verificación de roles antes del consumo de caché, mitigando accesos indebidos y reduciendo la carga en la nube.

2. La fachada AgroLabFacade resolvió el problema de omisión de cobros en los clientes móviles, centralizando las reglas de negocio en un único punto de orquestación mantenible.

3. La integración combinada de Proxy y Facade demuestra que abordar los problemas arquitecturales desde la intención adecuada produce software robusto y extensible.

Referencias:

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

- Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.